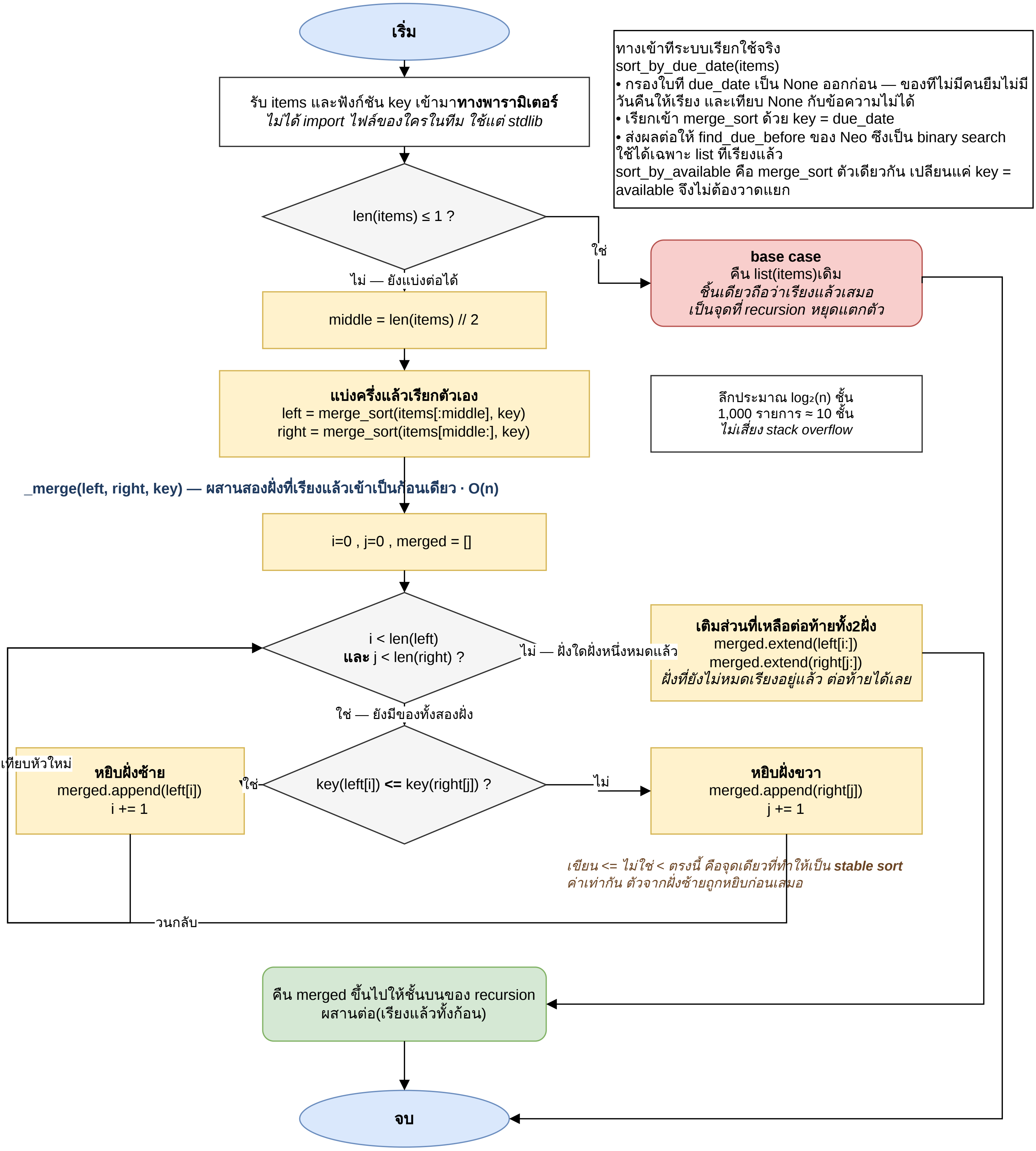
SO

วนกลับไป

# Flowchart 6 — การเรียงลำดับ: merge_sort (แบ่งครึ่ง → ผสาน)

rt_group.py · โครงเป็น recursion แบ่งครึ่งซ้ำจนเหลือชิ้นเดียว ต่างจาก hash_table.py ที่คำนวณตำแหน่งจาก key ตรงๆ และต่างจาก count_* ใน search.py ที่เดินข้อมูลรอบเดียวจบ


เริ่ม
รับ items และฟังก์ชัน key เข้ามาทางพารามิเตอร์
ไม่ได้ import ไฟล์ของใครในทีม ใช้แต่ stdlib
len(items) ≤ 1 ?
ใช่
ไม่ — ยังแบ่งต่อได้
base case
คืน list(items)เดิม
ชิ้นเดียวถือว่าเรียงแล้วเสมอ
เป็นจุดที่ recursion หยุดแตกตัว
middle = len(items) // 2
แบ่งครึ่งแล้วเรียกตัวเอง
left = merge_sort(items[:middle], key)
right = merge_sort(items[middle:], key)
ลึกประมาณ log₂(n) ชั้น
1,000 รายการ ≈ 10 ชั้น
ไม่เสี่ยง stack overflow
_merge(left, right, key) — ผสานสองฝั่งที่เรียงแล้วเข้าเป็นก้อนเดียว · O(n)
i=0 , j=0 , merged = []
i < len(left)
และ j < len(right) ?
ไม่ — ฝั่งใดฝั่งหนึ่งหมดแล้ว
เติมส่วนที่เหลือต่อท้ายทั้ง2ฝั่ง
merged.extend(left[i:])
merged.extend(right[j:])
ฝั่งที่ยังไม่หมดเรียงอยู่แล้ว ต่อท้ายได้เลย
ใช่ — ยังมีของทั้งสองฝั่ง
key(left[i]) <= key(right[j]) ?
ใช่
ไม่
หยิบฝั่งซ้าย
merged.append(left[i])
i += 1
หยิบฝั่งขวา
merged.append(right[j])
j += 1
เทียบหัวใหม่
เขียน <= ไม่ใช่ < ตรงนี้ คือจุดเดียวที่ทำให้เป็น stable sort
ค่าเท่ากัน ตัวจากฝั่งซ้ายถูกหยิบก่อนเสมอ
วนกลับ
คืน merged ขึ้นไปให้ชั้นบนของ recursion
ผสานต่อ(เรียงแล้วทั้งก้อน)
จบ
ทางเข้าที่ระบบเรียกใช้จริง
sort_by_due_date(items)
• กรองใบที่ due_date เป็น None ออกก่อน — ของที่ไม่มีคนยืมไม่มีวันคืนให้เรียง และเทียบ None กับข้อความไม่ได้
• เรียกเข้า merge_sort ด้วย key = due_date
• ส่งผลต่อให้ find_due_before ของ Neo ซึ่งเป็น binary search ใช้ได้เฉพาะ list ที่เรียงแล้ว
sort_by_available คือ merge_sort ตัวเดียวกัน เปลี่ยนแค่ key = available จึงไม่ต้องวาดแยก

**ทำไมแบ่งครึ่งแล้วผสาน ถึงเร็วกว่าการไล่สลับทีละคู่**

- แบ่งครึ่งซ้ำจนเหลือ 1 ทำให้มี **$\log_2(n)$ ชั้น** และการผสานแต่ละชั้นใช้ **O(n)** รวมเป็น **O(n log n)**
- bubble sort ต้องไล่เทียบทุกคู่ซ้ำไปเรื่อยๆ จึงเป็น **$O(n^2)$** — วัดจริงบนชุด D ขนาด 1,000 รายการ **merge sort เร็วกว่า bubble sort 34.6 เท่า**

- เขียน **<= ไม่ใช่ <** ตอนเทียบฝั่งซ้าย ทำให้เป็น **stable sort** ลำดับเดิมของรายการที่ due_date เท่ากันจึงไม่ถูกสลับมั่วโดยไม่มีเหตุผล
- คืน **list ใหม่เสมอ ไม่แก้ของเดิม** เพราะอาจมีคนอื่นถือ list ใบเดียวกันอยู่ แบบเดียวกับ sort_by_code ใน search.py
- list ที่เรียงแล้วจากไฟล์นี้คือตัวที่ **find_due_before (binary search ของ Neo)** ใช้ต่อโดยตรง ถ้าเรียงผิด การตัดครึ่งจะตัดผิดข้างโดย**ไม่มี error ฟ้องเลย**